

C++ 标准库简介

C++ 标准

首先需要介绍的是 C++ 本身的版本。由于 C++ 本身只是一门语言，而不同的编译器对 C++ 的实现方法各不一致，因此需要标准化来约束编译器的实现，使得 C++ 代码在不同的编译器下表现一致。C++ 自 1985 年诞生以来，一共由国际标准化组织（ISO）发布了 5 个正式的 C++ 标准，依次为 C++98、C++03、C++11（亦称 C++0x）、C++14（亦称 C++1y）、C++17（亦称 C++1z）、C++20（亦称 C++2a）。C++ 标准草案在 [open-std](https://ericniebler.com/open-std/) 网站上，最新的标准 C++23（亦称 C++2b）仍在制定中。此外还有一些补充标准，例如 C++ TR1。

每一个版本的 C++ 标准不仅规定了 C++ 的语法、语言特性，还规定了一套 C++ 内置库的实现规范，这个库便是 C++ 标准库。C++ 标准库中包含大量常用代码的实现，如输入输出、基本数据结构、内存管理、多线程支持等。掌握 C++ 标准库是编写更现代的 C++ 代码必要的一步。C++ 标准库的详细文档在 [cppreference](https://en.cppreference.com/) 网站上，文档对标准库中的类型函数的用法、效率、注意事项等都有介绍，请善用。

需要指出的是，不同的 OJ 平台对 C++ 版本均不相同，例如 [最新的 ICPC 比赛规则](#) 支持 C++20 标准。根据 NOI 科学委员会决议，自 2021 年 9 月 1 日起 [NOI Linux 2.0](#) 作为 NOI 系列比赛和 CSP-J/S 等活动的标准环境使用。NOI Linux 2.0 中指定的 g++ 9.3.0 [默认支持标准](#) 为 C++14，并支持 C++17 标准，可以满足绝大部分竞赛选手的需求。因此在学习 C++ 时要注意比赛支持的标准，避免在赛场上时编译报错。

标准模板库（STL）

STL 即标准模板库（Standard Template Library），是 C++ 标准库的一部分，里面包含了一些模板化的通用的数据结构和算法。由于其模板化的特点，它能够兼容自定义的数据类型，避免大量的造轮子工作。NOI 和 ICPC 赛事都支持 STL 库的使用，因此合理利用 STL 可以避免编写无用算法，并且充分利用编译器对模板库优化提高效率。STL 库的详细介绍请参见对应的页面：[STL 容器](#) 和 [STL 算法](#)。

► 什么是造轮子

Boost 库

[Boost](#) 是除了标准库外，另一个久副盛名的开源 C++ 工具库，其代码具有可移植、高质量、高性能、高可靠性等特点。Boost 中的模块数量非常之大，功能全面，并且拥有完备的跨平台支持，因此被看作 C++ 的准标准库。C++ 标准中的不少特性也都来自于 Boost，如智能指针、元编程、日期和时间等。尽管在 OI 中无法使用 Boost，但是 Boost 中有不少轮子可以用来验证算法或者对拍，如 Boost.Geometry 有 R 树的实现，Boost.Graph 有图的相关算法，Boost.Intrusive 则提供了一套与 STL 容器用法相似的侵入式容器。有兴趣的读者可以自行在网络搜索教程。

参考资料

1. [C++ reference](#)
2. [C++ 参考手册](#)
3. [维基百科 - C++](#)
4. [Boost 官方网站](#)
5. [Boost 教程网站](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[cmpute](#), [Enter-tainer](#), [Henry-ZHR](#), [lr1d](#), [opsiff](#), [real01bit](#), [Suyun514](#), [Tiphereth-A](#), [Xeonacid](#), [zeyu10](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用